

s4.05 — URL shortener

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Id generation and code encoding — IC3 baseline](#)
- [Cache tiers, redirect semantics, and abuse — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: first-hand 1p3a report retained in [the bank](#).
 - Prior notes marked this prompt as reported often as the *first* system design prompt of a loop.
 - Goal: a clean answer with numbers in about 25 minutes, then spend the saved clock on the deep dives the interviewer picks.
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- OCI framing for a business prompt
 - Control plane: link creation, edit, delete, takedown, and analytics reads.
 - Data plane: the public redirect.
 - Static stability: redirects must keep serving from caches and replicas while the control plane or the primary database is down.
 - Tenancy: API keys per tenant own links; the public code namespace is per hostname, not per tenant.
 - Blast radius: one viral code, one tenant's scan storm, or one region's cache loss must not take the redirect fleet down.
- Why this is hard
 - Naive: auto-increment id, base62 encode, done.
 - First break: nothing, for a while.
 - Writes are trivial.
 - Reads are `100-1000x` writes and long-tail.
 - The hard parts are `read latency` at scale, id generation without a single counter, and cache behavior for a zipfian read mix.
 - Second break: control.
 - A `301` that browsers cache permanently means you can never remap a code and you lose analytics.
 - A shortener is the canonical open-redirect phishing vector, so abuse control is part of the design, not an afterthought.
 - Minimum primitives

- A unique short code with no coordination bottleneck.
 - A read path that is cacheable to 90%+ hits with a hot-key tier.
 - A redirect status that keeps control (302/307) with click events off the hot path.
 - A kill switch per code.
- 🔊 "Writes are trivial and reads are a cache problem; the real design is id allocation without a hot counter and a redirect you can still take back."
- Opening script
 - "I will treat this as a read-heavy redirect data plane with a small control plane for link management."
 - "First I confirm scale, custom aliases, expiry, analytics freshness, and who may create links."
 - "Then I show the numbers: about 3 writes per second, 10k reads per second, 250 GB in five years, so one relational primary with replicas and a two-tier cache is enough."
 - "The deep dives are id generation with a leased-range allocator and base62, and the cache tiers with negative caching and hot-key absorption."
 - "I will close with 301 versus 302, abuse controls, the failure table, and where the design changes at 10x reads or multi-region writes."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Write and read rates, storage, code space; API semantics
Architecture	17:00–30:00	Diagram, hot path versus async versus control plane, one redirect trace
Deep dive	30:00–45:00	Range allocator and base62, cache tiers, 301 versus 302, abuse
Operations and tradeoffs	45:00–50:00	Failure table, hygiene, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.
 - Finish the baseline early on purpose.
 - The old budget was clarify 3, FR/NFR/BoE 5, HLD 6, API and schema 4, deep dives 8, wrap 2.
 - Saved minutes go to whichever deep dive the interviewer picks.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Scale?	100M new links per year (about 3/s write), 10k reads/s peak	Writes fit one primary; reads are a cache design
Custom aliases?	Yes, unique, first-come	A unique index is the arbiter; no reservation flow in v1
Expiry and delete?	Optional TTL; soft delete with tombstone	410 on read, lazy expiry plus sweep; deleted codes are never reused immediately
Analytics?	Click counts asynchronous, not real-time	Redirect never waits on analytics; hourly rollups are enough
Auth?	API keys per tenant for management; public unauthenticated redirect	Code uniqueness must be global per hostname because the redirect has no tenant context
Redirect contract?	Mutable destinations allowed	Forces 302/307 with bounded caching, not 301/308
Multi-region?	Single home region for writes; reads served locally	Keeps one write path at 3/s; replicas everywhere

- Functional scope
 - Create a link with a random code or a custom alias and an optional TTL.
 - Redirect on the public hostname.
 - Edit destination and expiry; soft delete.
 - Basic click statistics per link.
 - Creation-time and serve-time abuse controls.

Requirement	Target	Enforced by
Redirect latency	p99 < 20 ms	In-process LRU plus Redis in front of replicas; no synchronous analytics
Redirect availability	99.99%	Stateless redirect fleet; caches and replicas serve during primary or control-plane outage
Create latency	p99 < 100 ms	One primary write plus write-through cache; id block already leased locally
Code collisions	Zero, ever	Range-allocated counter ids plus a unique index on (domain, code)
Retention	5 y, about 500M rows	One relational primary with replicas; monthly partitions for click rollups
Cache hit ratio	≥ 90% on redirects	Zipfian read mix; negative caching; page when hit ratio falls below 80%
Analytics freshness	Within 1 h (hourly rollup)	Click consumer aggregates per code per hour from the event queue
Takedown propagation	Under 60 s end to end	Kill switch flips deleted, Redis DEL, pub/sub invalidation to node caches

- Invariants
 - (domain, code) is unique across all tenants on a shared public hostname.
 - Code uniqueness is committed before the short URL is returned.
 - A deleted or expired code is a tombstone; it is not reissued to a new owner while old bookmarks may exist.
 - Redirect success never depends on the analytics pipeline.
 - A generated-code collision retries allocation without changing user intent.
- Non-goals
 - Real-time analytics dashboards in v1.
 - Deduplicating identical destinations across owners.
 - Link previews or content proxying.
 - Multi-region write conflict resolution in v1.

Capacity and consequence — IC3 baseline

- Writes
 - $100M / y \div 3.15 \times 10^7 s \approx 3.2/s$ average; assume 10x peak, about 30/s.
 - Decision: one relational primary handles creates.
 - The id counter row is touched once per 100k ids, so at 3/s that is about once every 9 h.
 - No Snowflake and no dedicated id service.
- Reads
 - 10k reads/s peak with a zipfian mix.
 - Caching about 20% of keys yields 90%+ hits, so the database sees about 1k reads/s.
 - Decision: two cache tiers in front of replicas.
 - Replica count is sized for cache-miss traffic plus a cold-start burst, not for 10k/s.
- Storage
 - $500M \text{ URLs} \times \text{about } 500 \text{ B} \approx 250 \text{ GB}$ over 5 y.
 - Decision: one primary plus replicas holds this.
 - Sharding is optional; say so out loud.
 - Partition clicks_hourly monthly because it is append-heavy.
- Code space

- Base62 at length 7 gives $62^7 \approx 3.5 \times 10^{12}$ codes, far above 5×10^8 .
- Fill after five years is about 0.014%.
- Decision: seven characters, counter-based, no collision handling needed on the allocation path.
 - A truncated hash would collide: expected collisions at $n = 5 \times 10^8$ in $N = 3.5 \times 10^{12}$ are about $n^2 / 2N \approx 36,000$.
- Bandwidth
 - $10k/s \times 500 B \approx 5 MB/s$ of redirect responses.
 - Decision: a CDN is optional.
 - Edge caching helps tail latency, not capacity.
- Cache memory
 - Caching 20% of all rows would be 100M entries $\times$ about 300 B $\approx$ 30 GB, an order of magnitude estimate.
 - The real hot set is the last hours of traffic, far smaller.
 - Decision: cap Redis with `maxmemory` and `allkeys-lru` so the working set, not the table, sets cache size.
- Click events
 - Assume 3k/s average clicks, about 260M raw events per day.
 - Decision: raw clicks go to a queue and object storage, never to the relational store on the hot path.
 - The hourly rollup per code is tiny by comparison.
- So what
 - This is not a scale problem for writes or storage.
 - Say so in the first two minutes and spend the time on read latency, id allocation, and control.

Architecture and flow — IC3 baseline

```
[hot path / data plane]
GET /s/{code} -> LB -> redirect service -> local LRU (s1.03) -> Redis -> Postgres replica
      |
      +--> 302 Location: destination
      +--> click event -> queue                                [async]

[control plane]
POST /links -> link API -> id gen (leased range, base62) -> Postgres primary -> write-through Redis
PATCH/DELETE -> link API -> primary -> Redis DEL + pub/sub invalidate to node caches

[async]
click consumer -> aggregate per code per hour -> clicks_hourly; raw events -> object storage

[safety]
creation-time URL scanning + blocklist; serve-time kill switch per code (410, cache invalidated)
```

- Redirect service owns the data plane.
 - Mechanism bought: stateless nodes with an in-process LRU absorb hot keys and serve during control-plane outages.
 - Cost paid: node caches are eventually consistent; a delete is visible only after invalidation or local TTL.
- Redis owns the shared cache tier.
 - Mechanism bought: one shared lookup at about 2 ms keeps replicas at cache-miss traffic only.
 - Cost paid: a Redis outage shifts up to the miss ratio of the local tier onto replicas; singleflight is required.
- Relational store owns the source of truth.
 - Mechanism bought: the unique index on (domain, code) is the arbiter for custom aliases and generated codes.
 - Cost paid: single-region writes; replica lag can make a fresh code miss on a replica.
- Link API owns the control plane.
 - Mechanism bought: authenticated, tenant-scoped, idempotent creates and edits with write-through cache population.
 - Cost paid: every mutation pays a primary write and an invalidation fan-out.

- Id allocator owns uniqueness of generated codes.
 - Mechanism bought: each node leases a block of 100k ids from one counter row and encodes locally.
 - Cost paid: a crashed node wastes the rest of its block; codes are obfuscated so volume does not leak.
 - Click pipeline owns analytics.
 - Mechanism bought: append-only events aggregated per code per hour off the request path.
 - Cost paid: counts are eventually consistent and not billing-grade unless deduplicated by event id.
 - Safety controls own abuse.
 - Mechanism bought: blocklist and reputation check at create; kill switch at serve.
 - Cost paid: a reputation API on the create path adds latency and a dependency; fail policy must be explicit.
- Client requests GET /s/{code} on the public hostname.
- Load balancer routes to any redirect node.
- Node checks the in-process LRU; on hit it returns 302 Location: destination in about 1 ms.
- On miss it reads Redis; on hit it populates the LRU and returns.
- On Redis miss it takes the per-key singleflight and reads a replica by (domain, code).
- Unknown code caches a negative entry for 5 min and returns 404; deleted or expired returns 410.
- Found code is written to Redis with a 1 h TTL plus jitter and to the LRU.
- Node emits a click event to the queue asynchronously and never waits on it.
- Click consumer aggregates events into clicks_hourly; raw events land in object storage.

API and data model — IC3 baseline

Surface	Contract	Idempotency	Conflict/error behavior
POST /v1/links	{destination, customCode?, expiresAt?}; returns 201 {linkId, shortUrl}	Idempotency-Key per tenant; same key and payload returns the same link	409 alias taken; 422 blocked or disallowed scheme; same key with different payload returns 409
GET /s/{code} (public)	302 Location: destination	Read	404 unknown; 410 expired or deleted; Cache-Control bounds caching
HEAD /s/{code} (public)	Same redirect headers, no body	Read	Define analytics handling for bots and link previews
GET /v1/links/{id}	Management metadata plus ETag	Read	404 for another tenant's id, not 403
PATCH /v1/links/{id}	{destination?, expiresAt?}; returns 200 link	If-Match on ETag	412 on stale ETag; mutable destinations require bounded redirect caching
DELETE /v1/links/{id}	Soft delete; returns 204	Idempotent (already deleted returns 204)	Tombstone instead of immediate code reuse
GET /v1/links/{id}/analytics?from=&to=	Aggregated counts plus watermark	Read	Bounded range; counts need not be exact billing data

- Path note
 - The API companion spells these as /links and /s/{code}.
 - The interview shorthand below is the same contract.

```

POST /api/v1/urls {long_url, custom_alias?, ttl_days?} Idempotency-Key: uuid
→ 201 {code, short_url} 409 alias taken 422 blocked-url
GET /{code} → 302 Location: long_url (404 unknown, 410 expired/deleted)
DELETE /api/v1/urls/{code} → 204 (soft)
GET /api/v1/urls/{code}/stats → {clicks_total, by_day[...]}
```

State	Key	Fields/invariant
Link	(tenant_id, link_id)	Owner, domain, code, destination, state, expires_at, version; UNIQUE (domain, code)
Code lookup	(domain, code)	Global arbiter for custom aliases and generated codes; tombstone on delete
Idempotency record	(tenant_id, idempotency_key)	Request hash and link_id; retained at least the client retry horizon
Id range lease	range_start	leased_to, leased_at; one row per 100k block
Cache entry	(domain, code)	destination, state, expires_at, version; positive TTL 1 h plus jitter, negative TTL 5 min
Click rollup	(tenant_id, link_id, bucket_start, coarse_region)	click_count ≥ 0; hourly buckets; monthly partitions
Raw click event	event_id	link_id, time bucket, coarse region; no raw IP or full user agent retained beyond need

```

CREATE TABLE urls (
  code          varchar(10) PRIMARY KEY,          -- base62; unique per public hostname
  domain        text NOT NULL DEFAULT 'sho.rt',
  link_id       uuid NOT NULL,
  long_url      text NOT NULL,
  tenant_id     uuid,
  owner_id      text,
  version       bigint NOT NULL DEFAULT 0,        -- ETag for PATCH If-Match
  created_at    timestamptz NOT NULL DEFAULT now(),
  expires_at    timestamptz,
  deleted       boolean NOT NULL DEFAULT false
);
CREATE UNIQUE INDEX urls_alias ON urls (domain, code); -- custom aliases share the namespace
CREATE INDEX urls_owner ON urls (tenant_id, owner_id, created_at DESC);

CREATE TABLE id_ranges (
  range_start bigint PRIMARY KEY,
  leased_to   text NOT NULL,
  leased_at   timestamptz NOT NULL
);

CREATE TABLE clicks_hourly (
  code varchar(10), hour timestamptz, clicks bigint,
  PRIMARY KEY (code, hour)
);

```

- Schema notes
 - The companion schema keys links by (tenant_id, link_id) with UNIQUE (domain, code); the table above is the interview-shorthand equivalent.
 - Tenant-scoped code uniqueness is insufficient on a shared public hostname.
 - The redirect has no authenticated tenant context to disambiguate.
 - clicks_hourly is partitioned monthly in production.
 - Raw click events keep {eventId, linkId, timeBucket, coarseRegion} only.

Id generation and code encoding — IC3 baseline

- Options
 - Hash the URL: dedupe-friendly; collision handling needed on truncation.
 - Random plus retry: simple; a unique-index conflict retries.
 - At 0.014% fill the retry rate is negligible; the honest reason to prefer ranges is determinism and no conflict loop, not collision risk.
 - Counter plus base62: no collisions; the counter must scale.

- Pick: central counter becomes a range allocator.
 - Each app node leases a block of 100k ids from one database row.
 - Encode to base62 locally.
 - Obfuscate with a bijective bit-mix so codes do not reveal volume.
 - XOR with a constant or multiplication by an odd constant modulo 2^{64} are bijections.
 - A non-bijective mix reintroduces collisions.
- Concurrency and failure
 - Two nodes can compute the same `max(range_start)` and race on the primary key.
 - The loser gets a unique violation and retries the lease.
 - The primary key is the correctness boundary; the retry is the recovery.
 - A node crash wastes the rest of its block.
 - Acceptable at 62^7 headroom; say it.
 - A block lease is durable before any id from it is handed out.
 - Never reissue a range after a crash.
- Consistency boundary
 - Creation is strongly consistent: the unique index commits before the short URL is returned.
 - Redirects tolerate eventual consistency.
 - Say early: "redirects tolerate eventual consistency; creation can be strongly consistent, a cheap win."

```

ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

def encode62(n):
    if n == 0:
        return ALPHABET[0]
    out = []
    while n:
        n, r = divmod(n, 62)
        out.append(ALPHABET[r])
    return "".join(reversed(out))

class IdBlock:
    """Lease [start, start+size) from the DB; refill when exhausted."""
    def __init__(self, db, node_id, size=100_000):
        self.db, self.node_id, self.size = db, node_id, size
        self.next = self.end = 0

    def _lease(self):
        for _ in range(5):
            # PK conflict = another node won the same start
            try:
                return self.db.one(
                    """INSERT INTO id_ranges (range_start, leased_to, leased_at)
                    VALUES ((SELECT coalesce(max(range_start),0)+%s FROM id_ranges), %s, now())
                    RETURNING range_start""", (self.size, self.node_id)).range_start
            except UniqueViolation:
                continue
        raise RuntimeError("id range lease contended")

    def next_id(self):
        if self.next >= self.end:
            self.next = self._lease()
            self.end = self.next + self.size
        n, self.next = self.next, self.next + 1
        return n

# Node crash wastes the rest of its block – acceptable (62^7 headroom). Say it.

```

```

public final class Encoder {
    private static final char[] A =
        "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ".toCharArray();

    public static String encode62(long n) {
        if (n == 0) return "0";
        StringBuilder sb = new StringBuilder();
        for (; n > 0; n /= 62) sb.append(A[(int) (n % 62)]);
        return sb.reverse().toString();
    }
}
// Range lease mirrors the Python: one conditional INSERT ... RETURNING per 100k ids
// -> the counter row is touched 3/s ÷ 100k ≈ never; no bottleneck, no Snowflake needed.

```

- Exact failure
 - Truncated hash at 500M rows: about 36,000 expected collisions by the birthday bound; handle or avoid.
 - Unobfuscated counter: competitors read your daily volume from consecutive codes.
 - UUID codes: 22+ characters defeats "short".

Cache tiers, redirect semantics, and abuse — IC3 baseline

- Cache tiers
 - In-process LRU (s1.03, about 1 ms).
 - Redis (about 2 ms).
 - Replica (about 5–10 ms).
 - Cache-aside with a 1 h TTL plus jitter.
 - The value carries `destination`, `state`, `expires_at`, and `version`, so a 410 is served from cache too.
- Negative caching
 - Unknown codes from bot scans are cached as 404 for 5 min.
 - Without it a scan storm is a database storm.
 - Defeater: a freshly created code must not be poisoned by a stale negative entry.
 - Create writes through to Redis and deletes any negative entry, so the code resolves before replicas catch up.
- Delete and remap
 - Write-through delete: Redis `DEL` plus a pub/sub invalidate to node caches.
 - Bounded staleness equals the local TTL.
 - A remapped destination follows the same path and bumps `version`.
- Hot key
 - A viral link is absorbed by the per-node LRU by design.
 - That is the argument for the local tier.
- Stampede and cold start
 - `getOrLoad` singleflight per key from s1.03 coalesces concurrent misses.
 - Warm from the access-log top-k after a cache restart.
- 301 versus 302
 - Documented HTTP behavior (RFC 9110 §15.1): 301 and 308 are heuristically cacheable; 302 and 307 are cacheable only with explicit freshness.
 - Browsers keep a 301 for a long time in practice, so a 301 means you lose analytics and can never remap the code.
 - Choose 302 (or 307 to preserve the method) with `Cache-Control: private, max-age=0` or a short `max-age`.
 - If pure redirect performance mattered more than control, 301 plus a long CDN TTL.
 - `HEAD` and known preview bots get the same redirect but are excluded or flagged in analytics.
- Abuse and safety
 - Creation-time: blocklist and reputation API check; 422 on known-bad.

- Fail policy for a reputation outage is explicit: fail open with async rescan, or fail closed for anonymous tiers.
 - Serve-time: kill switch (deleted flag to 410, cache invalidated) for takedowns.
 - Rate limit creation per key (s1.01); CAPTCHA on the anonymous tier.
 - Open-redirect phishing is *the* abuse of shorteners.
 - Saying it unprompted signals production experience.
- Expiry
 - Lazy on read: 410 plus a tombstone.
 - A daily sweep tombstones expired rows.
 - This is the s1.03 TTL pattern applied to the store.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Redis unavailable	All redirect misses from local LRU	Redis error rate, replica QPS jump	Local LRU serves hits; singleflight bounds replica load	Restore Redis; warm from access-log top-k
Primary database down (control plane / metadata store)	Creates, edits, deletes, takedowns	Write error rate, health check	Redirects continue from caches and replicas (static stability)	Promote a replica; replay idempotent creates
Replica lag after create	Fresh codes on one replica	Lag metric, fresh-code 404 rate	Write-through Redis serves the new code; negative TTL is short	Lag alarm; route misses for codes younger than lag to the primary
Viral hot key	One code, one node's CPU	Per-key top-k QPS	Per-node LRU absorbs it; no replica traffic	None needed; optional CDN for that code
Scan storm of random codes	Replica read capacity	404 rate spike	Negative cache 5 min; per-IP rate limit	Block the source; keep negative entries
Cache cold start	Replica read capacity	Miss ratio jump	Singleflight per key; shed to 503 above replica capacity	Warm from top-k; scale replicas
Click queue backlog or outage	Analytics freshness only	Consumer lag	Redirect never waits; bounded local buffer then drop	Replay from raw event log in object storage
Phishing destination discovered	Users of one code	Reputation feed, abuse reports	Kill switch to 410 within 60 s	Rescan tenant's other links; revoke key if needed
Id range lease contention or node crash	One node's creates	Unique violations, lease retries	Retry lease; wasted block accepted	Nothing; headroom is 62 ⁷

- Observe
 - Redirect p99 and cache hit ratio (page below 80%).
 - 404 rate as a scan detector.
 - Create error rate and idempotency replays.
 - Replica lag and fresh-code miss rate.
 - Click consumer lag and dropped events.
 - Takedown propagation time.
- Retry discipline
 - Clients retry creates with the same Idempotency-Key and exponential backoff with jitter.
 - Redirect nodes retry a replica read once, then serve 503 rather than pile on.
 - Click emission is fire-and-forget with a bounded local buffer; never retry on the request path.
 - A 409 alias conflict or 422 blocked URL requires a corrective action, not a retry.
- Tenant isolation

- API keys map to one tenant; management ids from another tenant return 404, not 403, to avoid existence leaks.
- Per-tenant create quotas and rate limits cap abuse and id-block consumption.
- The public code namespace is per hostname; a custom domain per tenant gives a private namespace.
- One tenant's viral link cannot starve another's redirects because the local LRU absorbs it and the fleet is stateless.
- Rollout and rollback safety
 - Schema changes follow expand, migrate, contract; existing codes must decode forever.
 - Changing the obfuscation or encoding is a new code generation, never a rewrite of issued codes.
 - Cache keys carry a version so a value-shape change does not deserialize old entries.
 - Canary the redirect fleet; 301 versus 302 behind a flag with a short max-age during the canary.
 - The id allocator never reissues a leased range across a deploy or rollback.
- Hygiene
 - Database failover drill.
 - Archive raw events to object storage; partition clicks_hourly monthly.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Leased ranges plus base62	Counter blocks per node, encoded locally	No collisions, no hot counter, no retry loop	Wasted blocks on crash; must obfuscate	Baseline at any write rate
Random plus retry	Random 7-char code, unique index arbitrates	Stateless allocation, no volume leak	Retry loop; rate rises as space fills	Simplicity beats determinism
Hash and truncate	Hash the destination, take a prefix	Dedupe-friendly, deterministic	Birthday collisions at scale; must handle	Same URL must map to same code
Snowflake ids	Time plus node plus sequence	Sortable, distributed	Leaks timestamps; longer codes; more machinery	Already have a Snowflake service
Postgres plus replicas	One primary, read replicas, two-tier cache	Simple, 250 GB fits	Single-region writes	Baseline
KV store	Partitioned key-value (s3.04)	Horizontal writes, multi-region	Loses unique-index arbitration and SQL analytics	Over 10 TB or multi-region writes

Decision	Chosen	Alternative	Flips when
Ids	Leased ranges plus base62	Random plus retry	Write rate >> 10k/s (rare here)
Redirect	302	301 plus CDN	Analytics do not matter
Store	Postgres plus replicas	KV store (s3.04)	Over 10 TB or multi-region writes
Dedupe	None (each create is new)	Hash-based	Storage cost of duplicates matters

- IC4 stretch — 10× reads (100k/s)
 - CDN in front of redirects with a short TTL and purge hooks on delete or remap.
 - More replicas; same design.
 - The database still sees about cache-miss traffic only.
- IC4 stretch — multi-region
 - Reads: replicate everywhere and serve locally.
 - Writes: single home region at 3/s, or a per-region code prefix to avoid write conflicts.
 - Takedown must fan out to every region's cache; measure propagation per region.
- IC4 stretch — real-time analytics
 - Click events to a stream (s3.06) with 1 min windows.
 - Deduplicate by event_id if exact counts matter; separate human clicks from previews.
 - The redirect path never waits on analytics.

- IC4 stretch — cost and blast radius
 - Cell the redirect fleet and cache per region so a cache loss strands one region, not all.
 - Redis memory is the dominant cost; `maxmemory` with LRU keeps it to the working set.
 - Raw click retention in object storage is the second cost; set a retention window.
- Weakest assumption
 - A 90% cache hit ratio on a zipfian read mix.
 - A scan storm or a flat long-tail workload drops the ratio and moves 10k/s onto replicas.
 - Mitigation: negative caching, per-IP limits, and replica headroom sized for 50% hits.
- Over-engineering warning
 - Do not shard on day one; 250 GB is one database.
 - Do not build a reservation flow for custom aliases; the unique index is the arbiter.
 - Do not adopt Snowflake for 3 writes per second.
- Defeaters to volunteer
 - “Hash and truncate, collisions are rare” fails the birthday bound at scale.
 - “301 everywhere” means permanent cache and no control.
 - “Shard day one” ignores that 250 GB fits one database.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Scale, custom aliases, expiry, analytics freshness, global code namespace	Names the 301 versus 302 contract and the tombstone rule before being asked	States that writes are trivial and reads are the design
Estimates	3/s writes, 10k/s reads, 250 GB, 62 ⁷ codes	Birthday-bound collision count; Redis working-set sizing	A number kills sharding and Snowflake on the spot
Architecture	Hot path, async clicks, control plane split with one trace	Static stability of redirects during a primary outage; write-through against negative-cache poison	Every component has a mechanism and a price
Deep dive	Range allocator, base62, two-tier cache, negative cache	Lease race and PK arbitration; bijective obfuscation; RFC 9110 cacheability	Reasons about the exact failure, not the label
Operations	Failure table, kill switch, hit-ratio alarm	Rollout of encoding changes; per-region cells; takedown propagation SLO	Volunteers abuse and rollout before being asked
Wrap	Recap and 302 tradeoff	Names the hit-ratio assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: 301 or 302?
 - **IC3 baseline**
 - 302 (or 307).
 - A 301 is cached long-term by browsers, so you lose analytics and can never remap the code.
 - If pure redirect performance mattered more than control, 301 plus a long CDN TTL.
 - **IC4 stretch**
 - Per RFC 9110 301/308 are heuristically cacheable and 302/307 need explicit freshness, so send `Cache-Control` on every redirect and choose 308 only for a genuinely permanent contract.
- Q2: Why not UUID or Snowflake?
 - **IC3 baseline**
 - UUID is 22+ characters and defeats “short”.
 - Snowflake works but ids leak timestamps and it is more machinery than one leased-range row for 3 writes per second.

- **IC4 stretch**
 - The leased range is the same idea as a Snowflake sequence with the node id replaced by a durable lease; the difference is who owns uniqueness after a crash.
- Q3: 10x reads (100k/s)?
 - **IC3 baseline**
 - CDN in front of redirects with a short TTL and purge hooks, more replicas, same design.
 - The database still sees about cache-miss traffic only.
 - **IC4 stretch**
 - Size replicas for the worst plausible hit ratio, not the average, and cell the fleet per region.
- Q4: Multi-region?
 - **IC3 baseline**
 - Reads: replicate everywhere, serve locally.
 - Writes: single home region at 3/s, or a per-region code prefix to avoid write conflicts.
 - **IC4 stretch**
 - Takedown propagation becomes the cross-region SLO; measure it per region and fail the kill switch loudly if a region is unreachable.
- Q5: Analytics real-time?
 - **IC3 baseline**
 - Click events to a stream with 1 min windows.
 - The redirect path never waits on analytics.
 - **IC4 stretch**
 - Deduplicate by event_id, separate bots and previews, and label counts as non-billing unless deduplicated.
- Q6: Custom alias contention?
 - **IC3 baseline**
 - The unique index is the arbiter; 409 on conflict.
 - No reservation flow in v1.
 - **IC4 stretch**
 - If aliases become a product feature, add a short-lived reservation row with expiry rather than a distributed lock.
- Q7: Expiry?
 - **IC3 baseline**
 - Lazy on read (410 plus tombstone) plus a daily sweep.
 - Exactly the s1.03 TTL pattern.
 - **IC4 stretch**
 - Never reuse an expired or deleted code while old bookmarks may exist; reuse is a phishing vector.
- Q8 (premise reversal): Why keep a database at all? Hash the destination, make it stateless.
 - **IC3 baseline**
 - The redirect still needs code to destination, so the store stays.
 - A truncated hash collides at the birthday bound, about 36,000 times at 500M rows, and the collision handler is a database anyway.
 - **IC4 stretch**
 - Deterministic hashing also breaks per-owner analytics, expiry, and takedown because two owners share one code; dedupe is a feature to add behind the store, not a reason to remove it.
- Validation scenarios
 - Create with a lost response and an identical retry returns the same code.
 - Two creates with the same custom alias; exactly one wins.
 - Redirect during a primary database outage.
 - Fresh code requested before replica catch-up with a stale negative entry.

- Viral link at 50k/s on one code.
- Random-code scan at 5k/s.
- Redis restart under peak load.
- Takedown of a phishing link; measure time to 410 on every node.
- Node crash mid-block; no reissued ids.
- Tenant A key on tenant B link id returns 404.

Recall card

- "Writes are trivial; reads are a cache problem; control is the design."
- "Leased-range base62 ids: no hot counter, no collisions, obfuscated so volume does not leak."
- "Two-tier cache with negative caching; the local LRU is the hot-key answer."
- "302 with async clicks keeps control; 301 gives it away forever."
- "Open-redirect phishing is the abuse; blocklist at create, kill switch at serve."
- 5 numbers: 3/s writes
 - 10k/s reads
 - $62^7 \approx 3.5T$ codes
 - 250 GB / 5 y
 - 90% cache hit
- 3 failures: cache stampede (singleflight)
 - Viral hot key (local LRU)
 - Scan storm (negative cache)
- 3 defeaters: truncate-hash
 - 301 everywhere
 - Premature sharding
- Done when: full answer in 25 min with the estimates from memory
 - encode62 and the range lease written cold
- Wrap-up script
 - "The design is leased-range base62 ids, a two-tier cache in front of relational replicas, 302 redirects with asynchronous click events, and abuse controls at create and serve time."
 - "The key invariant is that (domain, code) is unique and committed before the short URL is returned, and redirects never depend on the control plane or the analytics pipeline."
 - "The main tradeoff is 302 over 301: slightly more redirect traffic in exchange for analytics and the ability to remap or kill a code."
 - "My weakest assumption is the 90% cache hit ratio; I would validate the read distribution first and size replicas for 50% hits."

References

- [Ranked question and report provenance.](#)
- [OCI role framing.](#)
- [Detailed API and schema companion.](#)
- Sibling designs
 - [s1.03 thread-safe LRU cache](#): local tier, singleflight, TTL pattern.
 - [s1.01 distributed rate limiter](#): create-path limits.
 - [s3.04 metadata KV store](#): the store alternative.
 - [s3.06 durable event log](#): click stream for real-time analytics.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values, TTLs, and hit ratios remain proposed interview assumptions.

- [RFC 9110 — HTTP semantics](#): redirect status codes; §15.1 lists 301 and 308 as heuristically cacheable and omits 302 and 307.
- [PostgreSQL INSERT / ON CONFLICT](#): unique-index arbitration for aliases and range leases.
- Alex Xu, *System Design Interview* vol. 1, ch. 8: base62 and the 301 versus 302 discussion.
- Hello Interview URL shortener write-up: counter-based allocation and cache tiers.
- Engineering talks from bit.ly: hot-key handling and analytics separation (canonical pattern; specific talk not verified).